

Anticipated Q&A — Knowledge Points (EN / 中文)

Project: Scaling Laws and Accuracy–Efficiency Trade-offs in Modern ConvNets on CIFAR-10

Group 148: Boyi Shi · Puhao Zhu · Jinye Gong · DD2424

Sources: [REPORT_zh.md](#), [paper/main.tex](#), [logs/results.csv](#), [configs/default.yaml](#)

Use this sheet for **oral defense / Q&A** after the ~4 min presentation. Numbers are from **200 epoch**, **seed=42** unless noted.

How to read this document

Column	Meaning
Topic	Category
Question (EN)	Likely question in English
Answer (EN)	Short spoken answer
问题 (中文)	可能被问到的中文问题
回答 (中文)	简短口语回答
Who	Suggested primary responder

1. Project overview & research questions

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Goal	What is your project about?	We study accuracy vs. FLOPs for three ConvNet block families—VGG, depthwise separable, and ConvNeXt-style—on CIFAR-10 (32×32) , with matched compute budgets and scaling ablations (width, kernel, depth).	项目做什么？	在 CIFAR-10 上，在 匹配算力 前提下比较 VGG、深度可分离、ConvNeXt 三类 block 的 精度–FLOPs 权衡 ，并做 width/kernel/depth 缩放消融。	Any
RQ1	What is RQ1 and the answer?	RQ1: Does ConvNeXt beat a strong VGG baseline under matched FLOPs ? Answer: No. VGG wins all three tiers; ConvNeXt is 2–3 points lower everywhere.	RQ1 是什么？结论？	RQ1: 匹配 FLOPs 下 ConvNeXt 是否优于 VGG? 否 。三档 VGG 均为第一，CN 每档低约 2–3 点。	Boyi
RQ2	What is RQ2 and the answer?	RQ2: Which axis—width, kernel, depth, block type—gives the best marginal accuracy per FLOP ? Width helps until saturation (VGG peaks at w=64); kernel enlargement hurts (~3.7 pt drop k=3 → 11); depth matters little for CN vs. VGG/DW.	RQ2 是什么？结论？	RQ2: 哪个维度每多一 FLOP 收益最大？ Width 有效但会饱和（VGG 在 w=64 达峰）； 大核负收益 （k=3 → 11 约降 3.7 pt）； depth 对追上 VGG 帮助有限。	Jinye
Why CIFAR-10	Why not ImageNet?	Course scope + CIFAR-10 is the classic low-res (32×32) benchmark where ImageNet-tuned designs may not transfer (our kernel result supports this).	为什么只做 CIFAR-10?	课程范围 + 32×32 是检验「ImageNet 设计是否迁移到小图」的经典设置；我们的 kernel 结果支持 不宜照搬 。	Boyi

2. Dataset & training protocol

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Data split	How did you split the data?	CIFAR-10: 45k train / 5k val / 10k test . Val indices fixed with seed=42 , val_ratio=0.1 , saved to cifar10_val_indices.pt . Test is evaluated once after training ends—no test tuning.	数据怎么划分？	45k 训练 / 5k 验证 / 10k 测试 ；验证集索引 seed=42 固定保存； test 只在训练结束后评一次 ，不参与调参。	Jinye
Optimizer	What optimizer and schedule?	AdamW , lr=0.001, weight_decay=0.05; cosine annealing with 5-epoch linear warmup , min_lr=1e-6; 200 epochs , batch size 128 .	优化器和学习率？	AdamW (lr=0.001, wd=0.05) + 余弦退火 + 5 epoch warmup ; 200 epoch , batch 128 。	Jinye
Augmentation	What augmentation?	RandomCrop(32, padding=4) , horizontal flip, standard CIFAR-10 normalization. No MixUp / CutMix in the main study.	数据增强？	RandomCrop(padding=4) 、水平翻转、标准归一化；主实验 不用 MixUp/CutMix 。	Jinye

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Why no MixUp	Why disable MixUp and AMP?	To keep all 17 runs identical in training recipe—we wanted a clean architectural comparison , not the highest possible leaderboard score.	为什么关 MixUp/AMP?	让 17 组实验配方完全一致 ，做 干净的架构对比 ，不是刷榜。	Jinye
Model selection	How do you pick the checkpoint?	Best validation accuracy during training; test accuracy reported once on the held-out 10k test set.	怎么选模型?	按训练过程中 val 最高 存 checkpoint; test 只报一次 。	Jinye
Reproducibility	Single seed—is that enough?	Seed 42 for all runs. Family gaps are 2–3 points , much larger than typical CIFAR-10 seed noise (~0.3 pt), so rankings are robust ; absolute numbers may shift slightly with more seeds.	只有一个 seed 可靠吗?	全用 seed 42 ; 族间差 2–3 点 ，远大于常见种子波动 (~0.3 点)， 排序可靠 ；绝对值多种子可能略变。	Jinye
Overfitting	Does VGG overfit?	Train acc nears 100%; best val 93.92% , test 92.78% —mild gap. Val saturates around epoch ~100; best val near epoch 197.	VGG 过拟合吗?	train 接近 100%，val 93.92% 、test 92.78% ，有一定差距但 test 仍很高；约 100 epoch 后 val 趋于饱和。	Jinye

3. Model architectures (three families)

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
VGG block	What is your VGG baseline?	Conv–BN–ReLU stacks, three MaxPool stages, MLP head, dropout 0.5; tunable width . This is our [E]-level strong baseline.	VGG 基线结构?	Conv–BN–ReLU 堆叠，三次 MaxPool ，MLP 头，dropout 0.5；可调 width ；课程 E 级 强基线。	Boyi
Depthwise	What is the depthwise model?	Depthwise conv + 1×1 pointwise with BN/ReLU; same stage layout as VGG for fair comparison; tunable width and kernel_size .	Depthwise 是什么?	深度卷积 + 1×1 pointwise ，BN/ReLU；stage 布局与 VGG 对齐；可调 width 和 kernel 。	Boyi
ConvNeXt	What is your ConvNeXt-style block?	Stem + stages of large-kernel depthwise , LayerNorm2d , inverted bottleneck (1×1 expand–contract), GELU, residual; default k=7 , depths (2,2,2) at w=64.	ConvNeXt 块包含什么?	大核 depthwise 、 LayerNorm 、 倒瓶颈 1×1、GELU、残差；默认 k=7 ，depths (2,2,2) 。	Boyi
Normalization	Why BN on VGG/DW but LayerNorm on CN?	Follows common practice: BN in classic conv stacks; LayerNorm is part of the ConvNeXt modern block we ablate. Not a controlled single-variable swap across families.	为什么 VGG 用 BN、CN 用 LN?	各自遵循常见设计； 不是 三族之间只换归一化层的严格控制实验。	Boyi
Params vs FLOPs	Does VGG win because it has more parameters?	No . At w=64: VGG 2.20M params, DW 1.19M, CN 1.66M—DW has fewer params but still beats CN. On the Pareto plot , block design dominates, not param count alone.	VGG 赢是因为参数多吗?	不是 。w=64 时 DW 参数最少仍全面优于 CN； Pareto 上看 block 设计 比参数量更重要。	Puhao

4. FLOPs, fair comparison & metrics

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Why match FLOPs	Why can't you compare default width=64 accuracies?	At w=64, FLOPs are 309.5M (VGG) , 40.4M (DW) , 462.4M (CN) —up to 13× apart. Raw test acc cannot answer RQ1 ; we align budgets first.	为什么不能直接比默认 w=64?	同 w=64 时 FLOPs 差可达 13 倍 （DW 仅 VGG 的约 13%）， 不能直接回答 RQ1 ，必须先对齐算力。	Puhao
FLOPs tiers	What are the three FLOPs tiers?	Low ~150M , medium ~310M , high ~460M (±10%). We tune width per family to land in each tier.	三档 FLOPs 是多少?	低 ~150M、中 ~310M、高 ~460M (±10%)；主要通过调 width 对齐。	Puhao
FLOPs = 2×MACs	Why FLOPs = 2 × MACs?	One MAC = one multiply + one add = 2 FLOPs . thop reports MACs; we multiply by 2 to match common paper conventions.	为什么 FLOPs 是 2 倍 MAC?	一次 MAC = 一次乘法 + 一次加法 = 2 次浮点运算 ； thop 出 MAC，乘 2 与文献一致。	Puhao
How count FLOPs	How do you measure FLOPs?	Static estimate with thop on input [1, 3, 32, 32] via utils/metrics.count_flops() —same for all models.	FLOPs 怎么算?	用 thop 对输入 [1, 3, 32, 32] 做静态估计，所有模型同一工具、同一输入。	Puhao
Pareto	What is your Pareto frontier?	Plot of test accuracy vs. FLOPs for all 17 runs. Upper envelope is VGG + DW only; every ConvNeXt point is dominated (some VGG/DW point is cheaper and more accurate).	Pareto 前沿是什么?	17 次运行的 test acc–FLOPs 图； 上包络 只有 VGG 和 DW；CN 各点都被 支配 。	Puhao

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Non-dominated	What is your best model overall?	vgg_baseLine w=64: 309.5M FLOPs, 92.78% test (best val 93.92%)—global best in our study.	全局最优模型?	vgg_baseLine w=64: 309.5M FLOPs, test 92.78% (val 最高 93.92%) 。	Puhao

5. Headline numbers (memorize for Q&A)

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Matched tiers	Give matched-FLOPs results (low / med / high).	Low: VGG 92.05% > DW 90.86% > CN 88.85%. Med: VGG 92.78% > DW 92.18% > CN 90.34%. High: VGG 92.46% > DW 92.15% > CN 90.12%.	三档对齐结果?	低: 92.05 / 90.86 / 88.85. 中: 92.78 / 92.18 / 90.34. 高: 92.46 / 92.15 / 90.12 (均为 VGG>DW>CN) 。	Puhao
DW efficiency	How efficient is depthwise at medium tier?	DW at 279.6M FLOPs reaches 92.18% vs VGG 92.78% @ 309.5M —about 99.4% of VGG accuracy with ~10% fewer FLOPs .	中档 DW 效率?	279.6M 达 92.18% , VGG 309.5M 为 92.78% ——约 99.4% 精度、少约 10% FLOPs 。	Puhao
Width saturation	Why not use VGG w=80 for best accuracy?	w=80 (481.9M FLOPs) gets 92.46% — worse than w=64 (92.78%). More FLOPs ≠ higher accuracy on CIFAR-10 here.	为什么不用 VGG w=80?	w=80 (481.9M) 只有 92.46% , 低于 w=64 的 92.78% —— 更多 FLOPs 不等于更高精度 。	Jinye
Kernel scan	What happens when kernel size increases?	For CN (w=52) and DW (w=175), test drops monotonically as k goes 3 → 5 → 7 → 11. k=3 best; k=11 worst (~88.2% CN, ~88.5% DW)—about 3.7 pt loss vs k=3 on DW.	核变大会怎样?	CN/DW 的 test 随 k 单调下降; k=3 最优, k=11 最差 (约 88.2% / 88.5%) , 相对 k=3 约 降 3.7 点 。	Jinye
CN best kernel	Best ConvNeXt config you found?	k=3, w=52: 91.85% test @ 294.3M FLOPs—still below VGG med 92.78% .	ConvNeXt 最好配置?	k=3、w=52: 91.85% @ 294.3M——仍低于 VGG 中档 92.78% 。	Jinye
Depth	What did depth ablation show?	CN (3,3,3) w=43 → 90.27%; (1,1,1) w=64 → 89.16% (retrained 200 ep). Deeper slightly beats shallower CN, but both far below VGG/DW .	深度消融结论?	(3,3,3) 90.27%, (1,1,1) 89.16% (已补训满 200 epoch) ; 加深略好于变浅, 但都 远低于 VGG/DW 。	Boyi
Experiment count	How many experiments?	17/17 completed at 200 epochs : 9 tier-aligned (3×3), 7 kernel-related (incl. defaults), 2 depth variants.	一共多少实验?	17/17 , 均 200 epoch : 档位对齐 9 + kernel 相关 7 + depth 2。	Jinye

6. Mechanism & “why” questions

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Why CN loses	ConvNeXt is SOTA on ImageNet—why lose on CIFAR-10?	After three pooling stages, the feature map is only 4×4 . Receptive field is already saturated ; large kernels over-smooth local detail. ImageNet long-range tricks do not transfer to 32×32.	ImageNet 上很强, 这里为何输?	三次池化后特征图仅 4×4 , 感受野已饱和; 大核过度平滑 ; ImageNet 的长程设计 不适用于 32×32 。	Jinye
4×4 geometry	Explain the 4×4 feature map argument.	Input 32×32 → pool ÷2 three times → 32/8 = 4 spatial size. A k=11 kernel on 4×4 covers almost the whole map—little extra context, more smoothing/blur.	4×4 怎么来的?	32 经三次 ÷2 池化 → 4×4 ; k=11 几乎盖住整图, 难再增益上下文, 易过平滑。	Jinye
DW vs CN	Why does depthwise beat ConvNeXt but not VGG?	DW uses efficient 3×3 separable convs aligned with small maps; CN adds heavy blocks (large DW + LN + bottleneck) that cost FLOPs without matching VGG’s standard conv expressiveness on this dataset.	为什么 DW 强于 CN 但不如 VGG?	DW 用 3×3 可分离卷积, 更适合小图; CN 块更复杂 (大核+LN+倒瓶颈) , FLOPs 高但在 CIFAR-10 上表达力仍不如标准 VGG 卷积。	Boyi
Marginal width	Which family gains most from widening?	DW low → med: +1.04 pt per 100M FLOPs; VGG peaks at w=64 then negative margin w=64 → 80; CN improves with width but stays below VGG/DW in absolute acc.	加宽谁收益最大?	DW 从低档到中档边际收益最高; VGG 在 w=64 达峰后加宽 负收益 ; CN 加宽有提升但绝对精度仍最低。	Puhao

7. Design recommendations & conclusions

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Practical rules	What should I use on CIFAR-10?	<~180M FLOPs: VGG w=48 (92.05%). ~280–330M: VGG w=64 (92.78%) or DW w=175, k=3 (92.18%) if FLOPs-sensitive. >~430M: don’t scale further—w=80 hurts. Avoid k≥7 on DW/CN.	CIFAR-10 实用建议?	<180M: VGG w=48 ; ~300M: VGG w=64 或省算力用 DW w=175,k=3 ; >430M 不必再堆; DW/CN 避免 k≥7 。	Jinye

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Takeaways (3)	Three main takeaways?	(1) VGG best Pareto family, peak 92.78%. (2) Tight budget → DW + k=3 . (3) Don't copy ImageNet large-kernel ConvNeXt to 32×32.	三条结论?	(1) VGG 最优 , 最高 92.78%。(2) 算力紧用 DW+k=3 。(3) 勿把小图当 ImageNet 搬大核 CN 。	Jinye
More FLOPs	Does more compute always help?	No . Examples: VGG w=80 < w=64; high-tier CN 462M still 90.12% < med VGG; k=11 highest FLOPs in kernel scan, lowest accuracy.	算力越大越好吗?	不是 。例: VGG w=80 不如 w=64; 高档 CN 462M 仍低于中档 VGG; k=11 FLOPs 高但精度最低。	Jinye

8. Limitations & validity

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Limitations	What are the main limitations?	Only CIFAR-10; single seed ; static FLOPs (no GPU latency/energy); MixUp/AMP off —conclusions apply to our fixed recipe , not every training trick.	主要局限?	仅 CIFAR-10 ; 单 seed ; 静态 FLOPs (无延迟/能耗); 关 MixUp/AMP ——结论限于 本组统一配方 。	Any
Future work	What would you do next?	Multi-seed mean±std; measure latency/energy ; test transfer to slightly higher res (e.g. CIFAR-100, small ImageNet subsets).	后续工作?	多种子统计; 测 GPU 延迟/能耗 ; 在更高分辨率或 CIFAR-100 上验证迁移。	Any
thop vs real speed	Does fewer FLOPs mean faster on GPU?	Not always— memory access, kernel fusion, and depthwise efficiency differ by hardware. We report FLOPs for fair architectural comparison ; latency was out of scope.	FLOPs 少就一定更快吗?	不一定 ——还取决于内存访问、算子实现等; 我们用 FLOPs 做 架构公平对比 , 未测真实延迟。	Puhao
Interrupted run	Any failed runs?	Shallow CN d(1,1,1) first stopped at ~2 epochs (41.67% test); retrained 200 epochs on 2026-05-23 → 89.16% —report uses the completed run.	有失败实验吗?	浅层 CN (1,1,1) 曾中断只训 2 epoch; 2026-05-23 补训满 200 epoch 得 89.16%——报告用补训结果。	Jinye

9. Implementation & reproducibility

Topic	Question (EN)	Answer (EN)	问题 (中文)	回答 (中文)	Who
Code structure	Where is the training code?	<code>train/pipeline.py</code> , <code>train/trainer.py</code> ; config <code>configs/default.yaml</code> ; queue <code>configs/experiments.yaml</code> ; runner <code>scripts/run_experiments.py</code> .	训练代码在哪?	<code>train/pipeline.py</code> 、 <code>trainer.py</code> ; 配置 <code>configs/default.yaml</code> ; 队列 <code>experiments.yaml</code> ; <code>scripts/run_experiments.py</code> 顺序执行。	Jinye
Models code	Where are the three architectures?	<code>models/vgg_baseline.py</code> , <code>models/depthwise.py</code> , <code>models/convnext_style.py</code> .	三种模型在哪?	<code>models/vgg_baseline.py</code> 、 <code>depthwise.py</code> 、 <code>convnext_style.py</code> 。	Boyi
Results log	How do you aggregate results?	Each run appends to <code>logs/results.csv</code> ; figures via <code>scripts/generate_report_assets.py</code> → <code>figures/</code> and <code>REPORT_DATA_zh.md</code> .	结果怎么汇总?	写入 <code>logs/results.csv</code> ; <code>generate_report_assets.py</code> 生成图和 <code>REPORT_DATA_zh.md</code> 。	Jinye
E/D/B grades	How does this map to course grades?	[E] VGG baseline 92.78% + PyTorch pipeline. [D/C] modern recipe (warmup+cosine, AdamW) + width scaling + LayerNorm in CN. [B/A] 17-run fair comparison, ablations, Pareto, design rules.	对应课程哪几档?	[E] 强 VGG 基线; [D/C] 现代训练配方 + width 缩放; [B/A] 17 组公平对比、消融、Pareto、设计规则。	Any

10. Team responsibilities (if asked “who did what”)

Member	Main contribution (EN)	主要负责 (中文)
Boyi Shi	Block implementations (<code>models/</code>), architecture variants, kernel/depth ablations	三种 block 实现、架构变体、kernel/depth 消融
Puhao Zhu	FLOPs counting, tier alignment, Pareto plots, figures	FLOPs 统计、档位对齐、Pareto 与图表
Jinye Gong	Training pipeline, experiment queue, logging, results summary, discussion	训练管线、实验队列、日志与结果汇总、讨论

Quick reference — numbers to memorize

Item	Value
Best test accuracy	92.78% (vgg_base line w=64, 309.5M FLOPs)
Experiments completed	17/17 × 200 epochs , seed 42
FLOPs tiers	~150M / ~310M / ~460M
Default w=64 FLOPs (misaligned)	VGG 309.5M , DW 40.4M , CN 462.4M
Med tier test (VGG / DW / CN)	92.78% / 92.18% / 90.34%
DW efficiency @ med	99.4% of VGG acc, ~90% FLOPs
Kernel k=3 → k=11 (DW)	92.18% → 88.49% (~3.69 pt)
VGG w=64 → w=80	92.78% → 92.46% (worse)

Speaker cheat sheet (who answers what)

Category	Primary
RQ, block design, VGG/DW/CN structure, depth/kernel architecture	Boyi
FLOPs, MACs, tiers, Pareto, tables/figures, params	Puhao
Training recipe, seed, 17 runs, MixUp off, pipeline, limitations	Jinye

Related files: [PPT_SCRIPT_en.md](#), [PPT_SCRIPT_zh.md](#), [REPORT_zh.md](#), [paper/main.tex](#)